위상수학적 음악 분석 — Step 4

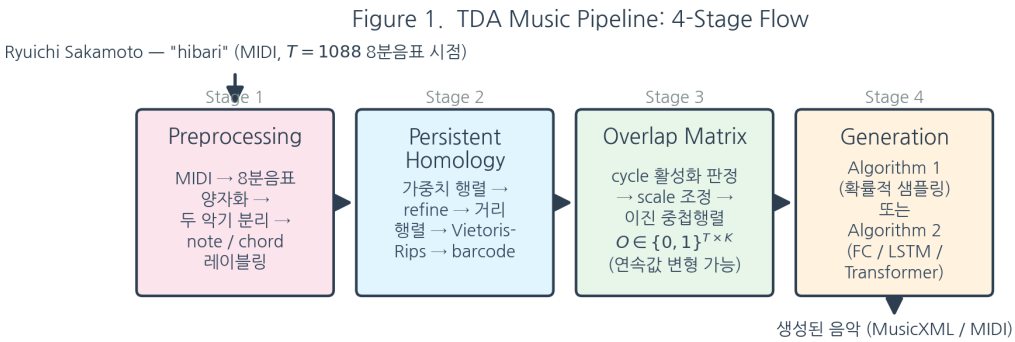
시각자료 (Figures)

본 장은 논문 본문에 삽입될 정적 figure 6개와 보충자료(supplementary material)로 제공되는 1개의 interactive figure를 모아서 제시한다.

본 장에 등장하는 시각자료의 개요

#	내용	배치
1	4-stage 파이프라인 흐름도	본문
2	발견된 cycle의 3D interactive 시각화 (재생 가능)	**보충자료**
3	Tonnetz 격자에 hibari 음 배치	본문
4	Persistence barcode 다이어그램	본문
5	원곡 vs 생성곡 piano roll 비교 (3 구간)	본문
6	학습 epoch별 JS divergence 곡선	본문
7	Inst 2의 모듈 구조 + 쉼표 시각화 (모듈 확대 포함)	본문

Figure 1 — TDA Music Pipeline: 4-Stage Flow



캡션. 본 연구의 파이프라인은 네 개의 순차적 단계로 구성된다. **Stage 1**은 MIDI 원본을 8분음표 단위로 양자화하고 두 악기를 분리한 뒤 note / chord 레이블링을 수행한다. **Stage 2**는 note 간 가중치 행렬을 구축하고 refine 과정을 거쳐 거리 행렬을 만든 뒤, Vietoris-Rips 복합체로부터 1차 persistent homology를 계산한다. **Stage 3**은 발견된 모든 cycle의 활성화 정보를 시점별 이진 중첩행렬 $O \in \{0,1\}^{T \times K}$로 변환한다(연속값 변형도 정의되어 있으나 본 실험에서는 이진 형태가 주로 사용되었다). **Stage 4**는 이 중첩행렬을 seed로 하여 Algorithm 1 (확률적 샘플링) 또는 Algorithm 2 (FC / LSTM / Transformer 신경망)로 새로운 음악을 생성한다.

Figure 2 (Interactive Supplementary) — hibari Cycles in 3D, with Click-to-Play

> 본 figure는 정적 PNG 형태로는 제공되지 않으며, 별도의 interactive HTML 파일로 제공된다 (제출 시 부록 디스크 또는 URL로 동봉).

이 figure를 정적 이미지에서 제외한 이유는 다음과 같다. 발견된 cycle은 46개에 달하며, 각 cycle은 여러 note를 공유하기 때문에 임의의 한 시점·각도에서 본 PNG로는 어떤 cycle이 어떤 note들을 공유하는지 거의 식별할 수 없다. 위에서 내려다보는 각도, 옆에서 보는 각도, 회전하면서 보는 동작이 모두 필요하다.

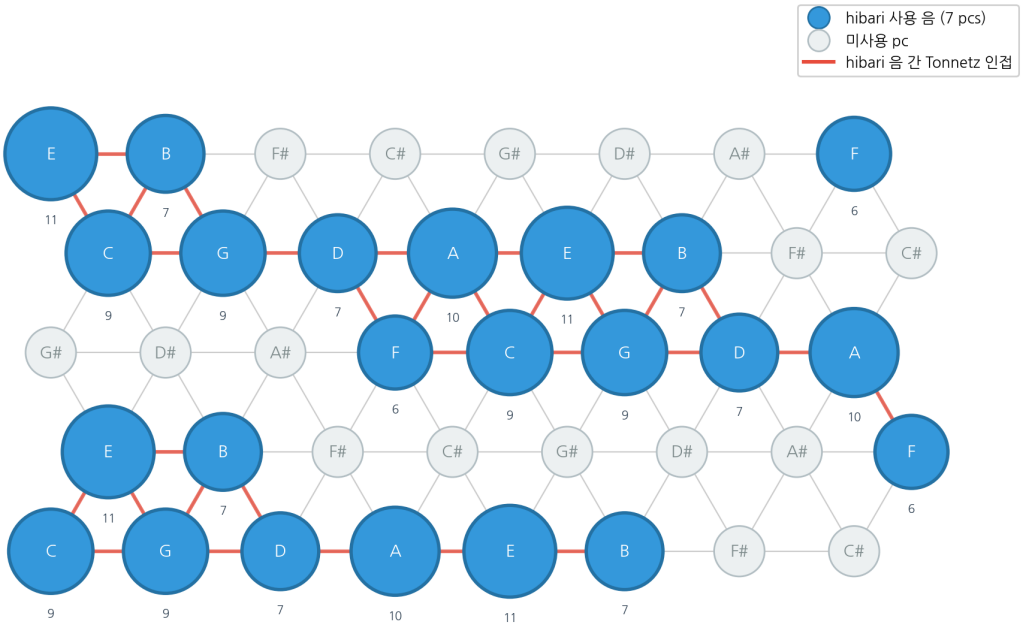
interactive HTML에서 가능한 조작.

- 마우스 드래그로 자유 회전, 스크롤로 줌
- 우측 legend 클릭으로 특정 cycle on/off
- cycle 선에 마우스를 올리면 그 cycle이 포함하는 모든 note의 pitch 이름 + 옥타브 + duration이 hover로 표시됨 (예: `label 19: C5 (dur=1)`)
- 화면 하단의 **"오디오 켜기"** 버튼을 한 번 클릭하면 브라우저 오디오 컨텍스트가 활성화되고, 이후 **cycle 선을 클릭하면** 해당 cycle을 구성하는 note들이 차례대로 sine wave로 재생됨 (Tone.js 기반, MIDI pitch → 주파수 변환). 이를 통해 "위상학적으로 한 cycle을 이루는 note들이 음악적으로 어떻게 들리는가"를 직접 청취할 수 있다.

상위 20개 cycle을 색상별로 구분하며, 카메라는 처음부터 위에서 내려다보는 각도로 설정되어 있다.

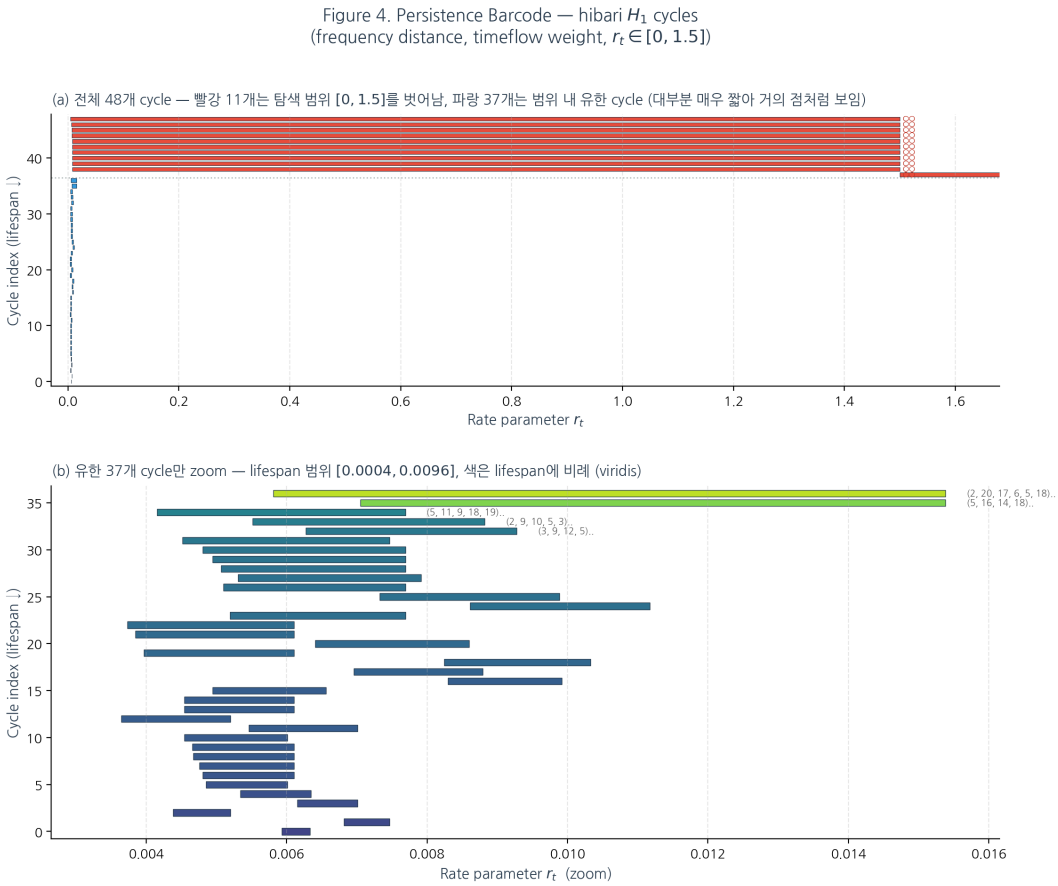
Figure 3 — Tonnetz 격자 위 hibari의 pitch class

Figure 3. Tonnetz 격자에 배치된 hibari의 7개 pitch class (C major scale)
빨간 선 = Tonnetz 그래프 상의 직접 인접, 숫자 = 원곡 내 출현 횟수



캡션. 12개의 pitch class가 배치된 Tonnetz 격자(가로 = 완전 5도, 대각선 = 장 / 단 3도) 위에서 hibari가 실제로 사용하는 7개 pitch class — C, D, E, F, G, A, B (C major scale)를 진한 파랑으로 강조하였다. 각 노드의 크기는 곡 내 출현 빈도에 비례하며, 빨간 선은 사용된 pitch class 쌍 중 Tonnetz 그래프 상에서 직접 인접한(거리 1) 관계를 나타낸다. 사용된 음들이 격자 위에서 하나의 연결 성분을 이루며 집중되어 있음을 확인할 수 있는데, 이는 hibari가 음악이론적으로 잘 정의된 토너츠 영역 안에서만 작동하는 곡이며, 따라서 Tonnetz 거리 함수가 frequency 거리보다 이 곡의 위상 구조를 더 잘 포착한다는 3.1절의 결과와 일관된다.

Figure 4 — Persistence Barcode Diagram (frequency distance)



데이터 출처. 본 figure는 `pickle/h1_rBD_t_notes1_1e-4_0.0~1.5.pkl`의 birth/death 기록을 사용한다. 이 pkl은 본 연구의 기본 거리 함수인 **frequency distance** (가중치 역수, Section 2.4) 와 timeflow weight, $r_t \in [0, 1.5]$ 범위에서 계산된 결과이다. Tonnetz 등 다른 거리 함수의 barcode는 형태와 cycle 수가 다르며, Step 3 Experiment 1에서 정량 비교된다.

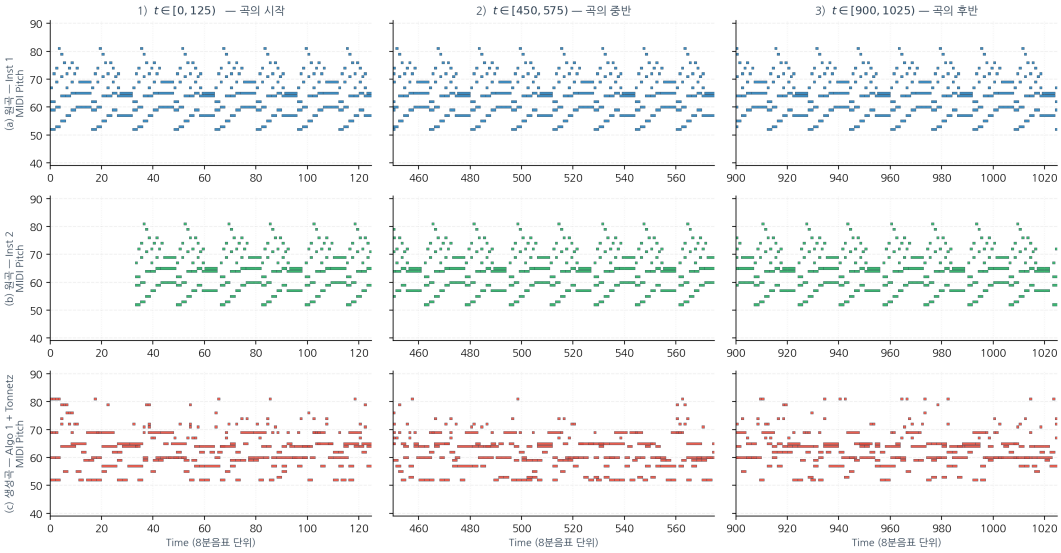
캡션. hibari의 H_1 cycle 전체 48개에 대한 persistence barcode를 두 panel로 나누어 보인다.

- **(a) 전체 barcode** — 가로축은 rate parameter r_t, 각 막대는 한 cycle의 $[\mathrm{birth}, \mathrm{death}]$ 구간이다. 빨강 11개는 탐색 범위 $[0, 1.5]$를 넘어서까지 살아남는 cycle (원본 death 값이 1.5를 초과; 막대를 $r_t = 1.5$에서 잘라내고 우측에 ∞ 표시). 파랑 37개는 탐색 범위 안에서 birth와 death가 모두 일어나는 유한 cycle인데, 빨강 cycle들에 비해 lifespan이 워낙 짧아 panel (a)에서는 거의 점처럼 보인다.
- **(b) 유한 cycle 37개만 zoom** — panel (a)에서 보이지 않던 짧은 cycle들의 lifespan 차이를 식별할 수 있도록 x축을 lifespan 실측 범위 $[0.0004, 0.0096]$로 확대하였다 (panel (a)의 약 $1/180$ 스케일). 색은 viridis colormap으로 lifespan에 비례하여 진해진다. 이 zoom을 통해 유한 cycle 사이에서도 약 24배의 lifespan 차이가 있고, 위쪽 5개 정도는 다른 유한 cycle들보다 명확히 더 안정적임을 확인할 수 있다.

이 두 그룹의 명확한 분리 자체 — 11개의 "탐색 범위를 넘어가는 안정적 cycle"과 37개의 "범위 안에서 birth와 death가 모두 일어나는 비교적 일시적인 cycle" — 가 hibari 위상 구조의 핵심 특징이며, 2.3절의 Elder Rule이 말하는 "구조적으로 의미 있는 cycle은 lifespan이 긴 것" 이라는 직관과 일관된다.

Figure 5 — Piano Roll 비교 (3 구간)

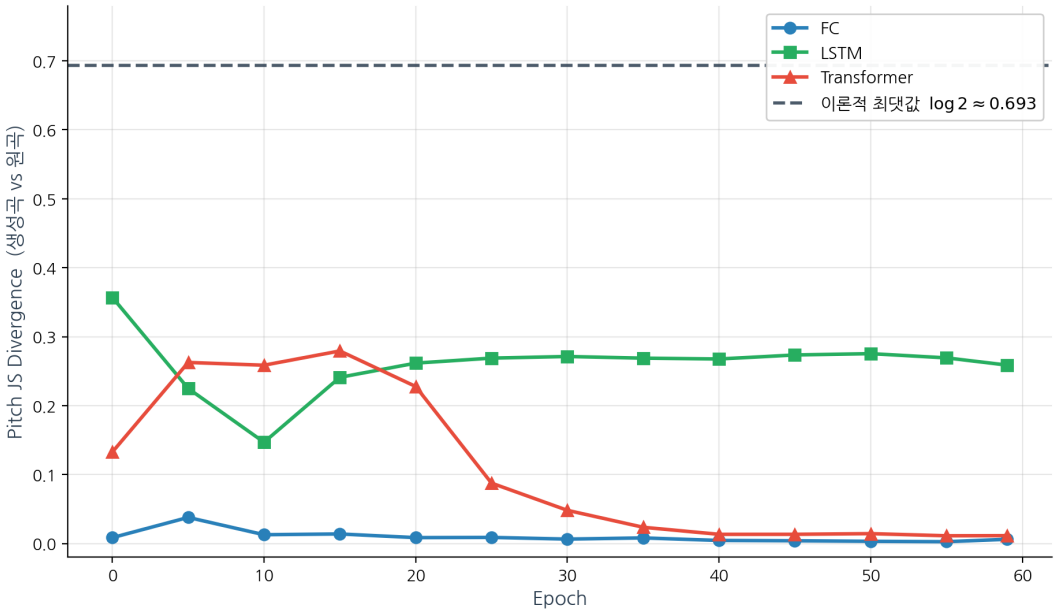
Figure 5. Piano Roll 비교 — 원곡 vs Tonnetz 기반 Algorithm 1 생성곡 (3개 구간 × 125 timesteps)



캡션. 위에서부터 순서대로 (a) 원곡 악기 1, (b) 원곡 악기 2, (c) Algorithm 1 + Tonnetz (K=46, seed=42)로 생성된 곡이다. 전체 $T = 1088$ timesteps을 한 번에 보이면 해상도가 낮아 개별 note가 식별되지 않으므로, 곡의 시작 ($t \in [0, 125)$), 중반 ($t \in [450, 575)$), 후반 ($t \in [900, 1025)$) 세 구간을 각 125 timesteps 씩 나란히 보여준다. 가로축은 시간(8분음표 단위), 세로축은 MIDI pitch이다. 각 구간에서 생성곡의 pitch 분포와 시간적 밀도가 원곡과 유사한 영역에서 활성화되고 있음을 시각적으로 확인할 수 있으며, 이는 3.1절의 JS divergence 정량 결과(0.0398)와 일관된다.

Figure 6 — 학습 Epoch별 JS Divergence 곡선

Figure 6. 학습 epoch별 JS Divergence — FC / LSTM / Transformer
(Tonnetz 기반 overlap, 동일 초기 seed)

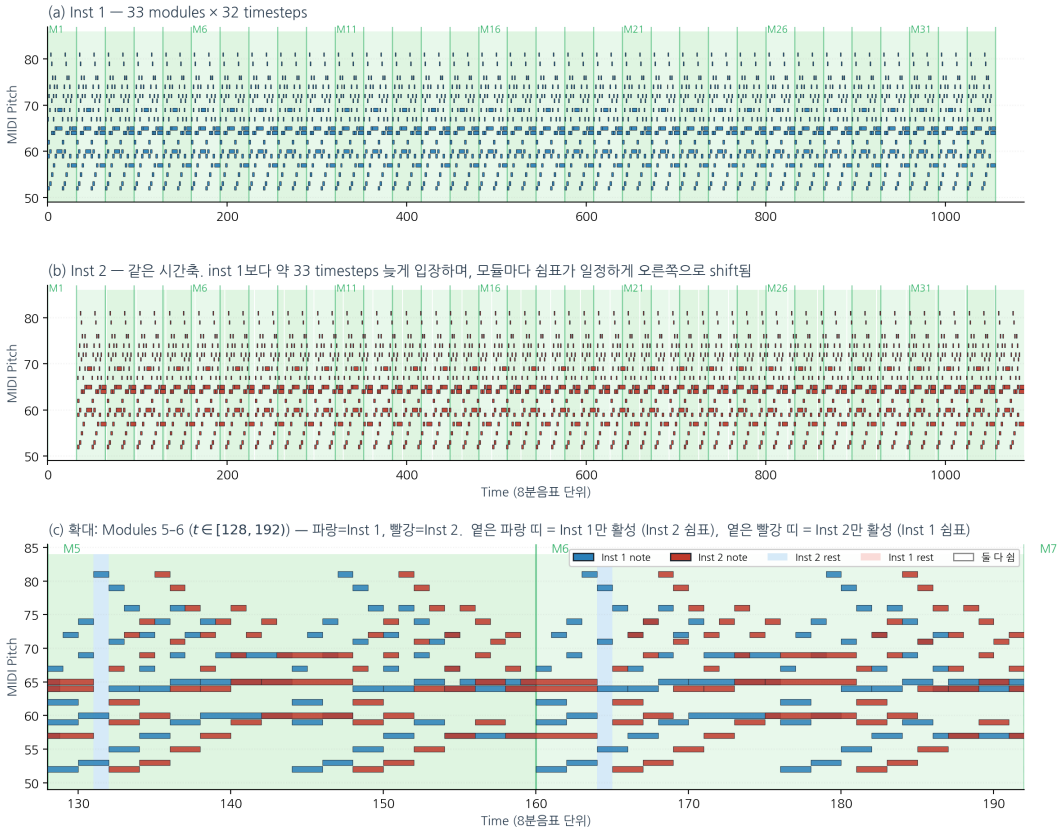


캡션. FC / LSTM / Transformer 세 모델을 동일한 데이터(Tonnetz overlap)와 동일 seed에서 60 epoch 학습시키면서, 매 5 epoch마다 생성곡을 만들어 원곡과의 pitch JS divergence를 측정한 결과이다. 진한 검정 파선은 이론적 최댓값 $\log 2 \approx 0.693$이다. FC 모델이 epoch 30 근처에서 $\mathrm{JS} \approx 0.003$에 도달하여 가장 빠르게 수렴하고, Transformer는 epoch 30 이후 같은 수준까지 따라잡는다. 반면 LSTM은 $\mathrm{JS} \approx 0.26$ 근처에서 plateau에 머물며 추가 개선이 일어나지 않는다. 이는 3.4절의 해석 — "hibari는 out of noise 앨범의 곡으로서

시간 문맥보다 음의 공간적 배치가 핵심이며, 시간 인과를 강하게 모델링하는 LSTM은 오히려 곡의 미학과 어긋난다" — 와 일관된 관찰이다.

Figure 7 — 두 악기의 모듈 구조와 쉼표 패턴 shift

Figure 7. 두 악기의 모듈 구조 — 쉼표 패턴이 modular operation처럼 shift



캡션. 본 figure는 hibari의 두 악기가 같은 모듈 구조를 공유하면서도, 쉼표 위치가 마치 modular operation처럼 일정하게 어긋나 있음을 시각적으로 드러낸다. 세 panel은 모두 같은 시간축 (8분음표 단위)을 공유한다.

- **(a) Inst 1 전체** — inst 1의 piano roll. 옅은 초록 배경은 32 timesteps 단위의 모듈 영역(짝수/홀수 모듈을 약간 다른 명도)이며, 진한 초록 수직선은 모듈 경계이다. 흰색 영역은 inst 1이 발성하지 않는 **쉼표** 구간이다. inst 1은 시점 $t = 0$에서 시작하여 $t = 1056$에서 끝난다.
- **(b) Inst 2 전체** — 같은 시간축 위 inst 2의 piano roll. inst 2는 $t = 33$에서 비로소 입장하여 $t = 1088$에서 끝난다 (즉 inst 1보다 약 33 timesteps 늦게 시작하고, 같은 양만큼 늦게 끝난다). (a)와 (b)를 위아래로 겹쳐 보면, 각 모듈 안의 쉼표 위치도 inst 1 대비 일정하게 오른쪽으로 shift되어 있음을 확인할 수 있다. 마치 모듈 길이를 modulus로 두는 modular operation을 가한 것과 같은 패턴이다 — 사카모토가 두 악기를 작곡할 때 한 악기를 시간 방향으로 충분히 채워 넣은 뒤 다른 악기를 그 위에 살짝 어긋나게 겹쳐 배치했다는 점(2.9절 인터뷰 참조)이 시각적으로 확인된다.
- **(c) Modules 5 - 6 확대** — 두 모듈 ($t \in [128, 192)$)을 확대하여 두 악기를 같은 그래프에 색을 달리해 겹쳐 표시한다. 파랑 사각형 = Inst 1 음표, 빨강 사각형 = Inst 2 음표. 배경 색의 의미는 다음과 같다:

- **옅은 파랑 띠** = inst 1만 활성, inst 2가 쉼표인 구간
- **옅은 빨강 띠** = inst 2만 활성, inst 1이 쉼표인 구간
- **흰색 영역** = 두 악기 모두 동시에 쉬는 구간

이 zoom에서, 모듈 안에서 두 악기의 쉼표가 서로 어긋나 있음, 그리고 한 악기의 쉼표를 다른 악기가 정확히 메꾸는 상보적 배치가 명확히 드러난다.

이 figure가 본 연구에서 중요한 이유는 두 가지이다. 첫째, 본 연구가 가중치 행렬을 처음부터 intra (각 악기 내부) / inter (두 악기 사이의 시차 결합) / simul (동시 발음) 세 가지로 분리한 설계가 단순한 수학적 편의가 아니라 **사카모토의 실제 작곡 방식에 근거한 결정**임을 시각적으로 정당화한다 (2.9절). 둘째, Algorithm 1이 사용하는 chord-height 패턴이 임의의 32-element 수열의 단순 반복이 아니라 **이 모듈 구조 자체**에서 추출된 것임을 보여준다 — 위상학적으로 모은 cycle 정보(중첩행렬)와 음악적 모듈 구조가 같은 시간축 위에서 결합되어 음악이 생성된다.

재현 방법

(전공자 / 코드 재현이 필요한 독자만 참조. 인쇄 제출본에는 본 절을 넣지 않는다.)

```
cd tda_pipeline/docs/figures

#    figures (      )
python make_fig1_pipeline.py        #
python make_fig3_tonnetz_hibari.py  #
python make_fig4_barcode.py         # pickle/h1_rBD_*.pkl
python make_fig5_pianoroll.py       #       + cache    (~5s)
python make_fig6_js_curve.py        # fig6_curves.json        ,
                                    #            (~2-3 min)
python make_fig7_inst2_modules.py   #           (~3s)

# Interactive supplementary
python make_fig2_interactive.py     # Plotly + Tone.js HTML
```

모든 정적 figure는 200 dpi PNG로 저장된다. 한글 폰트는 `_fontsetup.py`에서 NanumGothic을 자동 등록한다. Interactive figure는 `plotly` 패키지가 필요하며, 생성된 HTML은 외부 CDN에서 plotly.js와 Tone.js를 로드하므로 처음 열 때 인터넷 연결이 필요하다.